

QA AI Capability Exercise – Participant Guide

Build & Grow Your AI-Assisted Testing Workflow

A hands-on exercise every QA in the competency completes to strengthen and show how they work with AI for test design and automation. You'll get a feedback report and a personalized growth path – this is for development, not a graded test.

Contents

- > 1. What This Is
- > 2. Who Takes Part
- > 3. Time and Effort
- > 4. What You Get Out of It
- > 5. How the Exercise is Structured
- > Part A: AI Workflow Foundation
- > Part B: QA Mini Project (UI + API)
- > Submission Templates
- > Tool-Specific Expectations
- > What Counts as Complete
- > What Good Looks Like
- > Your Growth Path
- > Summary
- > Quick Tips

1. What This Is

This is a hands-on capability exercise to help you develop – and make visible – how you use AI tools effectively, responsibly, and practically across the **testing lifecycle**: requirement understanding, test planning, manual and automation test design, test data generation, execution, debugging, and documentation.

Everyone in the QA competency takes part; it is a shared part of how we build AI-assisted testing capability, not something a few people are singled out for. It is not a pass/fail exam – you will design and automate tests around a realistic application and show your thinking, and in return you get a feedback report and a clear sense of what to grow next.

What matters is not only whether the automation scripts run or the test cases exist, but **how you used AI** for requirement analysis, test strategy, prompt design, coverage decisions, debugging, and reflection. Making your testing thought process visible is the point.

2. Who Takes Part

All QA engineers and Leads in the competency – from junior to senior – who work with web UI and API-based applications, and commonly use tools such as Selenium, Playwright, Cypress, REST Assured, Postman, Karate, or similar.

The exercise is common across roles and stacks, but the **project option and testing depth** you may emphasize either UI-heavy, API-heavy, or balanced test coverage. Because everyone does it, there is a shared baseline and no one is measured against a different bar than their peers.

3. Time and Effort

The exercise is self-paced and meant to be completed within **one week**. You may work in any order; there is no required day-wise plan. Share your work by the agreed submission date.

Expected effort: the mandatory **Core QA project** is scoped for roughly **5–10** focused hours of work. The rest of the time goes into lifecycle artifacts – requirement analysis, prompt history, test design documents, execution and debugging logs,. Do not expand automation surface area at the expense of these artifacts.

4. What You Get Out of It

You receive a feedback report: your strengths, your growth areas, and concrete next steps for developing your AI-assisted testing workflow. The report also gives a sense of where you currently sit in the AI capability framework for QA and what would move you forward – think of it as a **snapshot and a direction**, not a grade.

Feedback focuses on areas like requirement analysis, prompting and context-setting, AI tool workflow, test strategy, test case depth, automation framework design, test data management, debugging, documentation, ownership, and responsible AI judgment – so you leave with a clear picture of what to practice next.

5. How the Exercise is Structured

It has three parts:

Part	Focus	Emphasis
------	-------	----------

----- ----- -----			
Part A	AI Workflow Foundation (for QA)	30%	
Part B	QA Mini Project (UI + API)	70%	

***** This Assignment is to be done in playwright using cursor AI . It has to be managed within the monthly limit.***

Part A: AI Workflow Foundation (QA)

Objective

Show that you understand how AI should be used in a **practical testing workflow** – thoughtfully, not as a simple “generate some test cases” shortcut.

Expected Submission

Submit a document named **project-info.md** covering:

- 1.What is project all about
- 2.Primary AI tool(s) used (e.g., ChatGPT, Cursor).
- 2.How you provide **project and system-under-test context** to the tool.
3. How you use AI for **requirement analysis** .
4. How you use AI for **test planning and strategy** (UI vs API, smoke vs regression, etc.).
5. How you use AI for **manual test case design** (functional, edge, negative, non-functional).
6. How you use AI for **automation design** (framework choice, structure, data, reusable utilities).
7. How you validate and refine **AI-generated test cases and scripts**.
8. How you use AI for **test data generation**, environment assumptions, and API payloads.

9. How you use AI for **debugging failing tests** and interpreting logs.
10. What information you avoid sharing unnecessarily with AI tools.
11. How you would **reuse this QA workflow** in a real project.

Part B: QA Mini Project – Manual + UI + API Testing

Objective

Demonstrate practical **AI-assisted QA delivery** through a realistic testing assignment around a small application. The project is split into UI and API Test Scenarios.

A **clean, well-documented Core** alone is a strong result. Both Core and Stretch are looked at the same way; the difference is the depth of evidence you show.

System Under Test (SUT):

1 . <https://practicesoftwaretesting.com/>

You will test a small ecommerce application , include all the possible flows that can be tested , categorize them as sanity or regression .

**** You need to press confirm twice to generate invoice**

A High Level Flow example

AC1: User Registration & Login

The user should be able to register with valid details, log in using the registered credentials, and verify their profile information successfully.

AC2: End-to-End Purchase Flow

The user should be able to browse products, add multiple items to the cart (including updating quantity), complete the checkout using Cash on Delivery, and successfully view the generated invoice under My Invoices. Suggestion to use like AC's

*** For Invoiceid press confirm button on application twice*

2. API Test - <https://api.practicesoftwaretesting.com/api/documentation>

You will test a flow/component in this activity.

##A High Level low example

AC1: User Authentication & Cart Creation

A new user should be able to register via API, log in with the registered credentials, obtain a valid bearer token, and create a new cart successfully.

AC2: Product Selection & Invoice Generation

Using the bearer token, the user should be able to retrieve products, add selected products to the cart, verify the cart contents, and successfully generate an invoice with the required customer and order details.

Example Request body for invoice generation post call

```
{
  "billing_street": "Zoey Shore",
  "billing_city": "Hesselbury",
  "billing_state": "Florida",
  "billing_country": "TG",
  "billing_postal_code": "1234AA",
  "payment_method": "cash-on-delivery",
  "cart_id": "01kx0dctdxxg6sm4wtt1t0nf9r",
  "payment_details": {}
}
```

}

> Submission Templates

Common QA Requirements (Regardless of Option)

Your submission must include:

1. **Requirement and risk analysis** specific to the application under test
2. A **project-info** document (Project Info , UI , API, positive/negative/edge ,Smoke /Regression).
3. A **manual test suite** for key flows (Manual/Functional Test case).
4. A **UI automation tier** (e.g., Playwright) covering smoke And E2E/regression flows.
6. A basic **API automation tier** (e.g., Playwright) covering core lifecycle APIs.
7. **Test data strategy** (how you design and/or generate data, including via AI).
8. Evidence of **test execution**: logs, reports, screenshots, or API collections.
10. README with **test setup and execution instructions**.
11. **Full prompt history** related to test design, automation and debugging.
12. All planning, design, testing, debugging, review, and reflection artifacts in a clear repository or folder structure.

Core Acceptance Criteria (QA Perspective)

Your Core submission should show that:

1. You can derive **clear test objectives and scope** from the application or tickets

2. You have a **traceable mapping** from requirements / state machine to test scenarios and cases.
3. You cover **valid and invalid status transitions** with tests (manual + API automation for the state machine).
4. UI tests verify key user flows: create, list, view, update, comment, search, error handling.
5. API tests verify key user flows: create, list, view, update, comment, search, error handling
6. Test data is well-planned (e.g., different priorities, statuses, edge case titles/descriptions).
7. At least one **automation suite** can be executed from the README without manual intervention beyond environment setup.
8. Prompt history shows **thoughtful AI use**, not copy-paste of unreviewed outputs.

Tool-Specific Expectations

- Use Playwright (Prism Framework) and Cursor

What Counts as Complete

Required Repository Structure--

qa-ai-practical-assessment/

- |— FunctionalTestCase (.csv)
- |— PrismStructure(Playwright/Selenium For API+UI+ Execution Report)
- |— project-info.md
- |— readme.md
- |— ai-prompts/
 - |— requirements-and-planning.md

- |— test-design.md
- |— automation-and-debugging.md
- |— [documentation-and-summary.md](#)

|— .Cursor/Tool

- Rules
- Skills
- agent/mcp (Optional)

****Project should include execution reports (execution reports) and status of all test cases should be 'Passed'**

**** Always follow iterative development methodology while prompting.**

**** Submit your assignment over public git and share the URL**

**** There should not be more than 5-8 test cases of each type (manual+UI+API) which includes @Smoke, @regression**

**** Git push should not be done in a single commit, it should be iterative development and push.**

File: project-info.md

Primary AI Tool(s) Used:

Application Under Test: PracticeSoftwareTesting Toolshop – Checkout & Application Flow

Assessment Start Date: / Submission Date:

Project Summary

(1–3 sentences describing the flow you tested and the main focus, e.g., new user checkout + invoice verification.)

Tools Used

(List browsers, automation tools, API tools, AI tools, and any supporting utilities.)

Setup Summary

1. How you provide **project and system-under-test context** to the tool.
2. How you use AI for **requirement analysis** .
3. How you use AI for **test planning and strategy** (UI vs API, smoke vs regression, etc.).
4. How you use AI for **manual test case design** (functional, edge, negative, non-functional).
5. How you use AI for **automation design** (framework choice, structure, data, reusable utilities).
6. How you validate and refine **AI-generated test cases and scripts**.
7. How you use AI for **test data generation**, environment assumptions, and API payloads.
8. How you use AI for **debugging failing tests** and interpreting logs.
9. What information you avoid sharing unnecessarily with AI tools.
10. How you would **reuse this QA workflow** in a real project.

File: [readme.md](#)

Project Information

What framework is used , How to run it ,where is test data and other fields required to run automation or manual test cases.

Include different test (Smoke ,Regression) command to run

Where the final reports are generated

AI Prompts Folder and History Expectations

Folder: `ai-prompts/`

Files:

text

`ai-prompts/`

|— `requirements-and-planning.md`

|— [`test-design.md`](#)

|— `test-data.md`

|— `automation-and-debugging.md`

|— `documentation-and-summary.md`

requirements-and-planning.md

text

AI Prompts – Requirements and Planning

(Record prompts and responses used for understanding the Toolshop flow, identifying risks, and drafting the test plan.)

For each entry:

- Prompt:
- AI Response (short summary):

test-design.md

text

AI Prompts – Test Design

(Prompts used to generate or refine test scenarios and test cases for UI + API.)

For each entry:

- Prompt:
- AI Response Summary:
- Validation Notes (how you checked coverage and correctness).

test-data.md

text

AI Prompts – Test Data

(Prompts used to generate test data for UI + API.)

For each entry:

- Prompt:
- AI Response Summary:
- Validation Notes (how you checked coverage and correctness).

automation-and-debugging.md

text

AI Prompts – Automation and Debugging

(Prompts used for automation structure, assertions, and analyzing failures/logs.)

For each entry:

- Prompt:
- AI Response Summary:
- Debugging Outcome (how it helped or misled you).

documentation-and-summary.md

text

AI Prompts – Documentation and Summary

(Prompts used for writing README, reports,)

For each entry:

- Prompt:
- AI Response Summary:
- Edits You Made:
- Reason for Edits (clarity, correctness, tone).

What Good Looks Like (QA)

Good submissions:

- Use AI to **augment** testing judgment, not replace it.
- Show a **clear, traceable line** from requirements to test design and automation.
- Demonstrate **iterative prompting** and careful review of AI output.
- Keep tests **maintainable and explainable**, not just “auto-generated”.
- Include **evidence of execution and logs**

Your Growth Path

The feedback report will indicate an **indicative standing** (e.g., Building the basics / Developing / Solid across the lifecycle / Advanced) and concrete coaching suggestions for growing your AI-assisted QA practice. Think of it as input into your normal growth track, not a final grade.

Summary

This exercise is about building and making visible your **AI-assisted testing workflow**: how you think, how you prompt, how you validate, and how you own the quality of the system under test. A smaller but well-tested Core with strong artifacts and reflection beats a large, superficially tested surface.

Quick Tips

1) Cursor model strategy (save tokens, stay within monthly limit)

Don't run the whole assessment on Sonnet 4.5/4.6. Use Auto by default, switch to Sonnet only when needed.

- Requirements, test design, risk analysis, ai-prompts docs → Auto / Composer 2.5 / lighter model → Why: planning & writing only; premium model not needed
- Page objects, spec files, playwright.config, API helpers, debugging failures → Sonnet 4.6 (or dedicated coding model) → Why: better code quality for Prism + Playwright JS
- Quick readme edits, CSV tweaks, folder structure fixes → Auto / Composer → Why: small edits; don't burn premium tokens

Rule: Auto for 70% of work → Sonnet for automation + hard debugging only.

2) Use "Caveman" skill — what & why

What: Caveman skill = keep prompts short & focused. One task per chat. No long story in every message.

Why useful for this assessment: • Less context per request → lower token usage • Faster responses in Cursor • Easier to copy real prompts into ai-prompts/ folder for evaluation • Avoids hitting context window mid-assessment

Project use cases: • "Extract AC1/AC2 from QA doc → list test scenarios" (separate chat) • "Generate FunctionalTestCase.csv rows for login negatives only" (separate chat) • "Create authApiPage.js following Prism pattern" (separate chat) • Don't mix all of the above in one giant thread

3) Summarize chat → save as .md (make a custom skill or Summarize Chat)

What: After a focused Cursor session, summarize the chat into a .md file (or use a custom skill: "summarize this chat into ai-prompts format").

Why useful: • Assessment needs prompt history — evaluators check how you used AI, not just final code • Long chats get lost; summary = clean evidence for ai-prompts/ • Frees context — start fresh chat for next phase without losing learnings

Project use cases: • After requirements chat → save summary to ai-prompts/requirements-and-planning.md • After test design chat → ai-prompts/test-design.md • After debugging failing TC-UI-16 (double confirm) → ai-prompts/automation-and-debugging.md • After test data / faker / invoice payload work → ai-prompts/test-data.md

Format per entry: Prompt → AI Response Summary → Validation Notes (what you changed & why)

4) Suggestion: Roughly/basic phase-wise flow for this project (Can differ)

Phase 1: QA doc + requirements + risk doc → Auto

Phase 2: Manual CSV + ai-prompts (record as you go) → Auto + summarize to .md

Phase 3: UI/API automation (PrismStructure-toolshop-playwright) → Sonnet

Phase 4: npm test smoke → full suite → execution-evidence screenshots → Auto/Composer 2.5/ GPT-5

Phase 5: Git push → done